

StudyPath

Product Requirements Document

Versão 1.0 · Maio 2025

Repositório	github.com/Studart7/StudyPath
Status	Em desenvolvimento ativo
Stack	React 18 + Node.js + PostgreSQL
Deploy FE	GitHub Pages (CI/CD automático)
Deploy BE	Local (cloud pendente)

Classificação: Interno · Documento vivo — sujeito a revisões

1. Visão Geral e Objetivo do Produto

1.1 O Problema

Estudantes autônomos e concurseiros frequentemente enfrentam a fragmentação do processo de aprendizado: anotações em cadernos, tarefas em apps de to-do genéricos, materiais espalhados em favoritos do navegador e sessões de estudo sem registro sistêmico. Essa desorganização gera perda de contexto, dificuldade em medir progresso real e falta de motivação por ausência de feedback visual.

1.2 A Solução

O StudyPath é uma plataforma web all-in-one que centraliza todo o ciclo de aprendizado em um único ambiente: definição de áreas de estudo, criação de tópicos com controle de progresso, gestão de tarefas vinculadas ao conteúdo, organização de materiais de referência e registro de sessões de estudo com cronômetro Pomodoro. O resultado é um painel de controle completo da jornada acadêmica do estudante.

1.3 Visão de Produto (Vision Statement)

Nossa Visão

"Ser a plataforma de referência para estudantes que desejam transformar disciplina em resultado, oferecendo clareza sobre onde estão, para onde vão e quanto progresso já fizeram — tudo em um único lugar."

1.4 Objetivos de Negócio

Objetivo	Métrica de Sucesso
Engajamento diário	≥ 3 sessões de estudo registradas por semana por usuário ativo
Retenção D30	≥ 40% dos usuários cadastrados ainda ativos no 30º dia
NPS	NPS ≥ 45 após 6 meses de operação
Performance técnica	P95 de resposta da API < 300 ms
Disponibilidade	Uptime ≥ 99,5% após deploy em nuvem

1.5 Fora do Escopo (MVP)

- Aplicativo mobile nativo (iOS/Android)
- Colaboração em tempo real entre estudantes
- Monetização / planos pagos
- IA generativa para geração de conteúdo de estudo
- Integração com plataformas de EAD (Coursera, Udemy)

2. Personas e Casos de Uso

2.1 Personas Primárias

Persona 1 — Lucas, o Concurseiro

Atributo	Detalhe
Idade	28 anos
Perfil	Pós-graduado, estuda para concurso público há 2 anos
Rotina	Estuda 6–8h/dia, divide o tempo entre múltiplas disciplinas
Dor principal	Dificuldade em saber qual matéria está mais defasada
Objetivo no app	Mapear todas as disciplinas, definir metas semanais e acompanhar progresso por tópico
Dispositivo	Desktop (notebook) — 80% do tempo
Critério de sucesso	Ver o heatmap de atividade cheio e a barra de progresso da área avançando

Persona 2 — Ana, a Universitária

Atributo	Detalhe
Idade	21 anos
Perfil	Estudante de Engenharia no 4º semestre
Rotina	Estuda por blocos entre aulas, usa o celular frequentemente
Dor principal	Esquece prazos de entrega e perde materiais de referência
Objetivo no app	Centralizar links, PDFs e tarefas de cada disciplina; receber alertas de prazo
Dispositivo	Mobile + Desktop
Critério de sucesso	Zero tarefas atrasadas no semestre

Persona 3 — Rafael, o Autodidata de TI

Atributo	Detalhe
Idade	34 anos
Perfil	Desenvolvedor que estuda novas tecnologias nas horas vagas
Rotina	1–2h de estudo por dia, foco em projetos práticos
Dor principal	Não sabe quanto tempo realmente investiu em cada tecnologia
Objetivo no app	Registrar sessões de estudo, medir horas por área e ver evolução ao longo do tempo
Dispositivo	Desktop — ambiente de trabalho

Atributo	Detalhe
Critério de sucesso	Relatório mostrando 100h acumuladas em Kubernetes em 3 meses

2.2 Casos de Uso Principais

#	Caso de Uso	Descrição
UC-01	Onboarding e cadastro	Usuário cria conta, confirma e-mail e cria sua primeira área de estudo
UC-02	Criar área de estudo	Usuário define nome, categoria, descrição e meta da área
UC-03	Gerenciar tópicos	Usuário adiciona tópicos, define dificuldade e atualiza progresso (0–100%)
UC-04	Registrar tarefa	Usuário cria tarefa com prazo, prioridade e vinculação a tópico
UC-05	Adicionar material	Usuário salva link/PDF/anotação associado a uma área ou tópico
UC-06	Sessão de estudo	Usuário inicia cronômetro Pomodoro, estuda e salva a sessão com notas
UC-07	Visualizar dashboard	Usuário consulta métricas: horas totais, streak, heatmap, progresso por área
UC-08	Alterar status de área	Usuário pausa, retoma ou conclui uma área de estudo
UC-09	Dark mode	Usuário alterna entre tema claro e escuro via toggle no header
UC-10	Excluir conta / dados	Usuário solicita exclusão dos seus dados (LGPD)

3. Requisitos Funcionais

3.1 Módulo de Autenticação

ID	Requisito	Prioridade	Status
RF-01	Cadastro de usuário com nome, e-mail e senha via Supabase Auth	Must Have	Implementado
RF-02	Verificação obrigatória de e-mail antes do primeiro acesso	Must Have	Implementado
RF-03	Login com e-mail e senha; JWT gerenciado pelo Supabase	Must Have	Implementado
RF-04	Logout que invalida a sessão do lado do servidor	Must Have	Implementado
RF-05	Fluxo de recuperação de senha ('Esqueci minha senha')	Must Have	Pendente
RF-06	Login social (Google OAuth)	Should Have	Pendente
RF-07	Exibição do perfil do usuário autenticado (/api/auth/me)	Must Have	Implementado

3.2 Módulo de Áreas de Estudo

ID	Requisito	Prioridade	Status
RF-10	Criar área com: nome, categoria, descrição, meta e status inicial	Must Have	Implementado
RF-11	Listar todas as áreas do usuário em cards com status visual	Must Have	Implementado
RF-12	Editar qualquer campo da área de estudo	Must Have	Implementado
RF-13	Excluir área (com confirmação e cascade em tópicos/tarefas/materiais/sessões)	Must Have	Implementado
RF-14	Alterar status: ativa pausada concluída	Must Have	Implementado
RF-15	Filtrar/buscar áreas por nome ou categoria	Should Have	Pendente
RF-16	Ordenar áreas por data de criação, progresso ou nome	Could Have	Pendente

3.3 Módulo de Tópicos

ID	Requisito	Prioridade	Status
RF-20	Criar tópico vinculado a uma área: nome, descrição, dificuldade, progresso	Must Have	Implementado

ID	Requisito	Prioridade	Status
RF-21	Listar tópicos de uma área com progresso visual (barra)	Must Have	Implementado
RF-22	Editar tópico (incluindo progresso 0–100% e status)	Must Have	Implementado
RF-23	Excluir tópico com confirmação	Must Have	Implementado
RF-24	Status automático: 'não iniciado' → 'em andamento' → 'concluído' conforme progresso	Should Have	Parcial
RF-25	Filtrar tópicos por dificuldade ou status	Should Have	Pendente

3.4 Módulo de Tarefas

ID	Requisito	Prioridade	Status
RF-30	Criar tarefa: título, descrição, prazo, prioridade, dificuldade, status	Must Have	Implementado
RF-31	Vincular tarefa opcionalmente a um tópico da mesma área	Must Have	Implementado
RF-32	Listar tarefas com badges de prioridade e indicador de prazo	Must Have	Implementado
RF-33	Atualizar status da tarefa: pendente em andamento concluída cancelada	Must Have	Implementado
RF-34	Excluir tarefa com confirmação	Must Have	Implementado
RF-35	Filtrar tarefas por status, prioridade ou prazo	Should Have	Pendente
RF-36	Notificação visual de tarefas próximas ao vencimento (≤ 2 dias)	Should Have	Pendente
RF-37	Notificações push/e-mail de lembretes de prazo	Could Have	Pendente

3.5 Módulo de Materiais

ID	Requisito	Prioridade	Status
RF-40	Adicionar material: título, tipo, URL, descrição, vínculo a área/tópico	Must Have	Implementado
RF-41	Tipos suportados: link, vídeo, livro, artigo, PDF, anotação, curso, outro	Must Have	Implementado
RF-42	Listar materiais com ícone por tipo e link clicável	Must Have	Implementado

ID	Requisito	Prioridade	Status
RF-43	Editar e excluir material	Must Have	Implementado
RF-44	Preview de URL (meta tags: título, thumbnail)	Could Have	Pendente
RF-45	Upload direto de PDF (armazenamento no Supabase Storage)	Could Have	Pendente
RF-46	Exportar lista de materiais como CSV ou PDF	Could Have	Pendente

3.6 Módulo de Sessões de Estudo

ID	Requisito	Prioridade	Status
RF-50	Registrar sessão: área, tópico (opcional), data, duração, nível de foco, dificuldade percebida, notas	Must Have	Implementado
RF-51	Cronômetro Pomodoro integrado: 25 min foco / 5 min pausa com alertas sonoros	Must Have	Implementado
RF-52	Salvar sessão automaticamente ao finalizar ciclo Pomodoro	Must Have	Implementado
RF-53	Listar histórico de sessões por área com filtro de data	Must Have	Implementado
RF-54	Excluir sessão registrada	Must Have	Implementado
RF-55	Duração personalizada do Pomodoro (configurável pelo usuário)	Should Have	Pendente
RF-56	Relatório de horas estudadas por semana/mês	Should Have	Pendente

3.7 Dashboard e Analytics

ID	Requisito	Prioridade	Status
RF-60	Exibir total de horas estudadas, streak atual e recorde de streak	Must Have	Implementado
RF-61	Heatmap de atividade (últimos 12 meses, estilo GitHub contributions)	Must Have	Implementado
RF-62	Cards de resumo por área: progresso, tarefas pendentes, última sessão	Must Have	Implementado
RF-63	Endpoint GET /api/dashboard com métricas agregadas	Must Have	Implementado
RF-64	Gráfico de horas por área (últimas 4 semanas)	Should Have	Pendente

ID	Requisito	Prioridade	Status
RF-65	Indicador de áreas sem atividade há mais de 7 dias	Should Have	Pendente
RF-66	Exportação do histórico completo em CSV ou PDF	Could Have	Pendente

4. Requisitos Não Funcionais

4.1 Performance

ID	Requisito	Meta
RNF-01	Tempo de resposta da API (P95)	< 300 ms para rotas GET; < 500 ms para POST/PUT
RNF-02	First Contentful Paint (FCP) do frontend	< 1,5 s em conexão 4G
RNF-03	Tamanho do bundle JS inicial	< 200 KB gzipped
RNF-04	Consultas ao banco de dados	Queries críticas com tempo de execução < 50 ms via índices
RNF-05	Carregamento do Dashboard	< 800 ms incluindo chamada ao /api/dashboard

4.2 Segurança

ID	Requisito	Implementação
RNF-10	Autenticação stateless com JWT	Supabase Auth — token não exposto no localStorage
RNF-11	Autorização por recurso (multi-tenant)	user_id em todas as queries; middleware valida token
RNF-12	Rate limiting global	200 req / 15 min por IP (express-rate-limit)
RNF-13	Rate limiting em rotas de auth	10 req / 15 min por IP
RNF-14	Limite de body size	10 KB — rejeita payloads maiores
RNF-15	SQL Injection	Prisma ORM com queries parametrizadas
RNF-16	Segredos e credenciais	.env no .gitignore; nunca expostos no repositório
RNF-17	Row Level Security (RLS)	Pendente — camada adicional no Supabase PostgreSQL
RNF-18	HTTPS obrigatório	Garantido via GitHub Pages (FE) e pelo provedor de nuvem (BE)
RNF-19	Headers de segurança (HSTS, CSP)	A configurar pós-deploy em nuvem

4.3 Escalabilidade e Disponibilidade

ID	Requisito	Meta / Abordagem
RNF-20	Uptime após deploy em nuvem	≥ 99,5% SLA mensal
RNF-21	Escalabilidade horizontal do backend	Stateless (JWT) — pronto para múltiplas instâncias
RNF-22	Connection pooling ao banco	Transaction Mode para runtime (Supabase pooler)
RNF-23	Suporte a usuários concorrentes iniciais	≥ 100 usuários simultâneos sem degradação
RNF-24	Backup de dados	Backup automático diário gerenciado pelo Supabase

4.4 Usabilidade e Acessibilidade

ID	Requisito	Meta
RNF-30	Design responsivo	Funcional em viewport ≥ 375 px (mobile) e ≤ 2560 px (4K)
RNF-31	Dark Mode	Toggle persistente no header; preferência salva em localStorage
RNF-32	Contraste de cores	WCAG AA — contraste ≥ 4,5:1 para texto normal
RNF-33	Navegação por teclado	Todos os elementos interativos acessíveis via Tab / Enter
RNF-34	Feedback de loading states	Skeleton screens ou spinners em toda chamada assíncrona
RNF-35	Mensagens de erro amigáveis	Nenhum erro técnico exposto diretamente ao usuário

4.5 Manutenibilidade

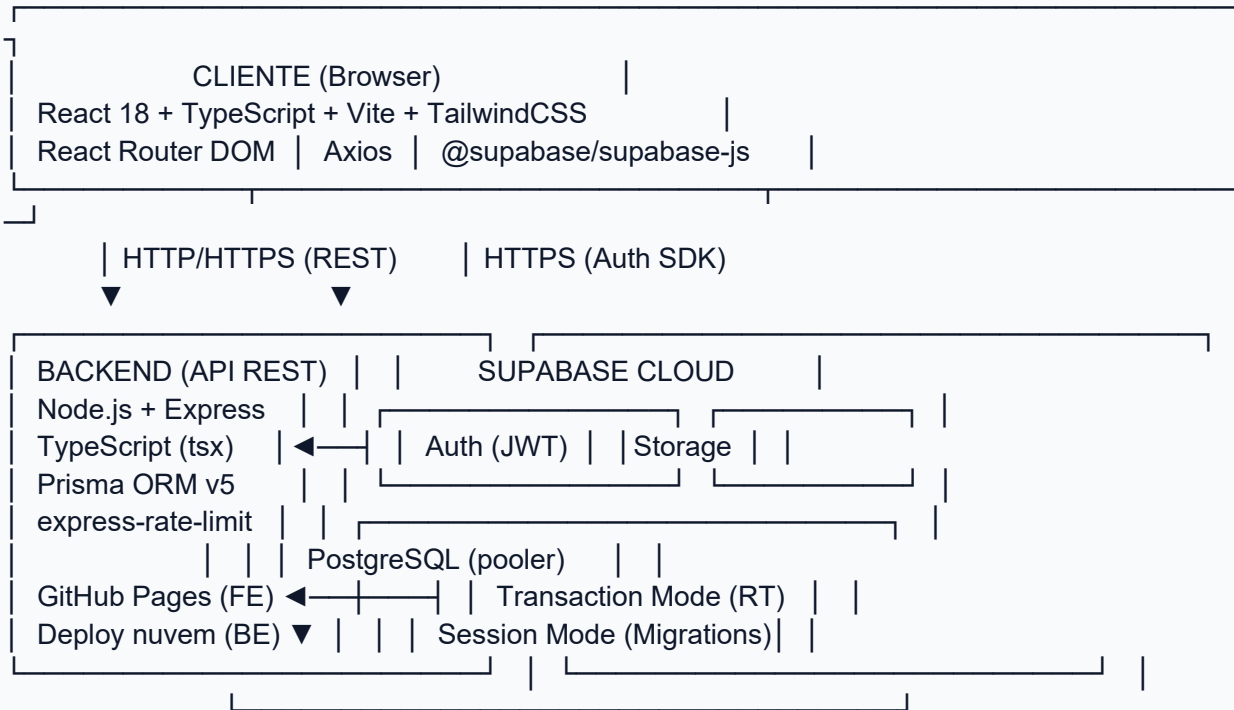
- Cobertura de testes unitários ≥ 70% nas funções críticas do backend
- TypeScript strict mode ativado em frontend e backend
- ESLint + Prettier configurados e executados no pipeline de CI
- Documentação da API via Swagger/OpenAPI (pendente)
- Commits seguindo Conventional Commits (feat, fix, docs, refactor...)

5. Arquitetura Técnica

5.1 Visão Geral da Arquitetura

O StudyPath adota uma arquitetura cliente-servidor desacoplada (SPA + API REST), com autenticação delegada ao Supabase Auth e persistência em PostgreSQL gerenciado pelo Supabase Cloud.

Diagrama de Arquitetura (texto)



CI/CD: GitHub Actions → build → deploy → GitHub Pages (FE)

5.2 Stack de Tecnologias

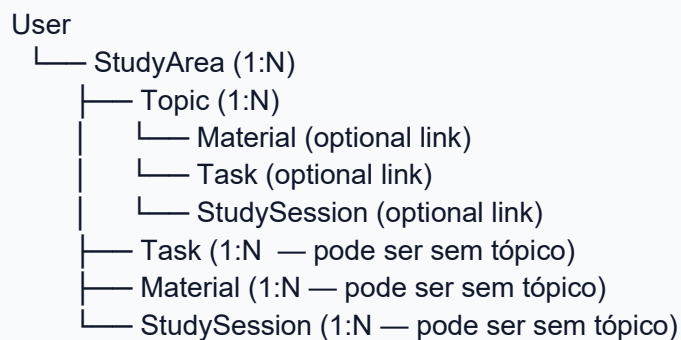
Camada	Tecnologia	Versão	Justificativa
Frontend — UI	React	18.x	Ecosistema maduro, hooks, concurrent features
Frontend — Types	TypeScript	5.x	Segurança de tipos, DX superior
Frontend — Build	Vite	5.x	HMR ultrarrápido, bundle otimizado
Frontend — Style	TailwindCSS	3.x	Utility-first, dark mode nativo, purge CSS
Frontend — Roteamento	React Router DOM	6.x	SPA routing, nested routes, loaders
Frontend — HTTP	Axios	1.x	Interceptors, cancel tokens, tipagem
Backend — Runtime	Node.js	20 LTS	Estabilidade, async nativo, amplo ecossistema

Camada	Tecnologia	Versão	Justificativa
Backend — Framework	Express	4.x	Minimal, flexível, vasto middleware
Backend — ORM	Prisma	5.x	Type-safe queries, migrations, studio GUI
Auth	Supabase Auth	2.x	JWT gerenciado, OAuth, email verification
Banco de Dados	PostgreSQL	15.x	ACID, JSON, full-text search, extensível
Deploy FE	GitHub Pages	—	Grátis, CI/CD integrado ao repositório
Deploy BE (alvo)	Railway / Render	—	Deploy simples de Node.js, free tier generoso

5.3 Modelo de Dados — Relações

Todas as entidades possuem `user_id` como chave de isolamento multi-tenant. As relações seguem a hierarquia abaixo:

Hierarquia de Dados



5.4 Decisões Arquiteturais (ADRs)

ADR	Decisão	Justificativa
ADR-01	Supabase Auth (não auth própria)	Elimina gerenciamento de passwords, MFA, email verification. Reduz superfície de ataque.
ADR-02	JWT não salvo em localStorage	Tokens ficam em memória/cookie HttpOnly via Supabase SDK. Mitiga XSS token theft.
ADR-03	Prisma ORM (não SQL raw)	Queries tipadas em TypeScript, migrations versionadas, proteção contra SQLi.
ADR-04	GitHub Pages para o FE	Zero custo, CI/CD integrado, HTTPS automático, ideal para SPA estática.
ADR-05	SPA (não SSR/SSG)	Simplicidade de deploy estático; SEO não é crítico para app autenticado.
ADR-06	Connection pooler (Transaction Mode)	Supabase limita conexões diretas. Transaction Mode multiplexa conexões para o runtime.

6. Fluxos de Usuário Principais

6.1 Fluxo de Onboarding

Etapa	Descrição
1	Usuário acessa a URL do app e é redirecionado para /login
2	Clica em 'Criar conta' → toggle exhibe formulário de registro
3	Preenche nome, e-mail e senha → clica em 'Criar conta'
4	Frontend chama Supabase Auth; Supabase envia e-mail de verificação
5	Usuário recebe e-mail, clica no link de confirmação
6	Supabase redireciona para a URL de callback; sessão JWT é estabelecida
7	Backend cria registro na tabela User (sincronização via webhook ou na primeira chamada /api/auth/me)
8	Usuário é redirecionado para o Dashboard — já autenticado
9	Modal/onboarding sugere criar a primeira área de estudo

6.2 Fluxo de Sessão de Estudo (Pomodoro)

Etapa	Descrição
1	Usuário acessa o Workspace de uma área de estudo
2	Navega para a aba 'Sessões'
3	Seleciona tópico (opcional) e nível de foco esperado
4	Clica em 'Iniciar Pomodoro' — cronômetro de 25 min começa
5	Ao fim dos 25 min: alerta sonoro e notificação visual
6	Formulário de encerramento aparece: notas, dificuldade percebida, foco real
7	Usuário clica em 'Salvar Sessão' — POST /api/sessions
8	Sessão aparece no histórico; heatmap e streak são atualizados no Dashboard
9	Opção de iniciar pausa de 5 min ou nova sessão

6.3 Fluxo de Gestão de Tópico

Etapa	Descrição
1	Usuário acessa o Workspace → aba 'Tópicos'
2	Clica em '+ Novo Tópico' → modal abre
3	Preenche nome, descrição e dificuldade → salva (POST /api/topics)
4	Tópico aparece na lista com barra de progresso em 0%

Etapa	Descrição
5	Após estudar, usuário clica no tópico → edita progresso para 60%
6	Status muda automaticamente para 'em andamento'
7	Ao atingir 100%, status muda para 'concluído'; ícone de check exibido

6.4 Fluxo de Autenticação (Login)

Etapa	Descrição
1	Usuário acessa a URL → redirecionado para /login (rota protegida)
2	Preenche e-mail e senha → clica em 'Entrar'
3	Frontend chama <code>supabase.auth.signInWithPassword()</code>
4	Supabase valida credenciais; retorna JWT (<code>access_token</code> + <code>refresh_token</code>)
5	Token é armazenado na sessão gerenciada pelo SDK (não <code>localStorage</code> manual)
6	Axios interceptor anexa Bearer token em toda requisição ao backend
7	Backend middleware valida token via <code>supabase.auth.getUser()</code>
8	Usuário é redirecionado para o Dashboard

7. Roadmap e Priorização do Backlog

7.1 Critérios de Priorização

Os itens do backlog são classificados pelo framework MoSCoW combinado com impacto estimado no usuário e esforço de implementação:

Categoria	Critério	Prazo Alvo
Must Have	Fundamental para o produto funcionar; bloqueia usuário sem isso	Sprint atual
Should Have	Alto valor, mas há workaround; deve ir na próxima sprint	Próxima sprint
Could Have	Agrega valor, mas não é urgente; entra se houver capacidade	Backlog médio prazo
Won't Have (now)	Fora do escopo atual; pode ser revisado no futuro	Não planejado

7.2 Roadmap por Fase

Fase 1 — Estabilização e Deploy (Sprint 1–2)

#	Item	Categoria	Esforço
1	Deploy do backend no Railway/Render	Must Have	M (2–3 dias)
2	Configurar variáveis de ambiente de produção	Must Have	P (< 1 dia)
3	Atualizar CORS e URLs do frontend para produção	Must Have	P (< 1 dia)
4	Ativar RLS (Row Level Security) no Supabase	Must Have	M (2 dias)
5	Implementar recuperação de senha	Must Have	M (1–2 dias)
6	Testes de integração nas rotas críticas da API	Should Have	G (3–4 dias)
7	Documentação OpenAPI/Swagger das rotas	Should Have	M (2 dias)

Fase 2 — Melhorias de UX e Features de Produtividade (Sprint 3–5)

#	Item	Categoria	Esforço
1	Filtros e buscas em áreas, tópicos e tarefas	Should Have	M (2 dias)
2	Notificação visual de tarefas vencendo (≤ 2 dias)	Should Have	P (1 dia)
3	Duração personalizada do Pomodoro	Should Have	P (1 dia)
4	Gráfico de horas por área (dashboard aprimorado)	Should Have	M (2 dias)
5	Indicador de áreas sem atividade (> 7 dias)	Should Have	P (1 dia)
6	Relatório semanal de horas estudadas	Should Have	M (2 dias)
7	Login social (Google OAuth via Supabase)	Could Have	M (2 dias)

Fase 3 — Funcionalidades Avançadas (Sprint 6–10)

#	Item	Categoria	Esforço
1	Notificações por e-mail (lembretes de prazo)	Could Have	G (3–5 dias)
2	Exportação de dados em CSV e PDF	Could Have	G (4 dias)
3	Preview de URL para materiais (Open Graph)	Could Have	M (2 dias)
4	Upload de PDF no Supabase Storage	Could Have	G (4 dias)
5	Gamificação: badges por streaks e metas atingidas	Could Have	G (5 dias)
6	Modo PWA (offline + instalável)	Could Have	G (5–7 dias)
7	Integração com Google Calendar	Won't Have	XG (10+ dias)
8	Compartilhamento de áreas entre usuários	Won't Have	XG (10+ dias)

8. Critérios de Aceitação por Feature

Cada feature é considerada 'Done' quando todos os critérios abaixo forem atendidos. Os critérios seguem o formato Gherkin (Given / When / Then).

8.1 Autenticação

Cadastro e Verificação de E-mail

DADO que um novo usuário acessa a tela de cadastro
QUANDO preenche nome, e-mail válido e senha ≥ 8 caracteres e clica em 'Criar conta'
ENTÃO um e-mail de verificação é enviado em até 30 segundos
E o usuário NÃO consegue fazer login antes de confirmar o e-mail
E após confirmar, o redirecionamento ocorre para o Dashboard em até 3 segundos

DADO que o usuário esquece a senha
QUANDO clica em 'Esqueci minha senha' e informa o e-mail cadastrado
ENTÃO recebe e-mail com link de redefinição em até 60 segundos [Pendente]

8.2 Sessão de Estudo / Pomodoro

Cronômetro e Salvamento

DADO que o usuário está no Workspace de uma área ativa
QUANDO inicia um Pomodoro
ENTÃO o cronômetro regride de 25:00 até 00:00 em tempo real
E ao chegar em 00:00 um alerta sonoro é disparado
E o formulário de encerramento exibe campos: notas, foco (1–5), dificuldade (1–5)

QUANDO o usuário salva a sessão
ENTÃO POST /api/sessions retorna 201 em < 500 ms
E a sessão aparece no histórico da área imediatamente
E o streak do Dashboard é incrementado se for o primeiro estudo do dia

8.3 Áreas de Estudo — CRUD

Criar e Excluir Área

DADO que o usuário está autenticado
QUANDO cria uma área com nome obrigatório preenchido
ENTÃO a área aparece na listagem em menos de 1 segundo
E o status inicial é 'ativa'

QUANDO tenta criar área com nome vazio
ENTÃO o formulário exibe erro de validação sem fazer chamada à API

QUANDO exclui uma área e confirma a ação
ENTÃO todos os tópicos, tarefas, materiais e sessões vinculados são removidos (cascade)

E a área desaparece da listagem em menos de 1 segundo

8.4 Deploy do Backend em Nuvem

Critérios de Aceite — Deploy

DADO que o backend está deployado em Railway/Render
QUANDO o frontend em produção faz uma chamada autenticada a /api/dashboard
ENTÃO a resposta chega em < 300 ms (P95)
E o status HTTP é 200 com payload JSON correto
E NENHUMA credencial ou stack trace é exposto no response body

QUANDO um usuário não autenticado tenta chamar /api/areas
ENTÃO a API retorna 401 Unauthorized

QUANDO um usuário tenta acessar área de outro usuário via ID
ENTÃO a API retorna 403 Forbidden ou 404 Not Found

8.5 Row Level Security (RLS)

Critérios de Aceite — RLS

DADO que RLS está ativado em todas as tabelas
QUANDO uma query ao banco não inclui o user_id correto
ENTÃO o Supabase rejeita o acesso e retorna 0 registros (não erro 500)

QUANDO um teste de penetração tenta acessar dados de outro usuário
ENTÃO nenhum dado de terceiro é retornado, mesmo com token válido de outro usuário

9. Riscos e Mitigações

9.1 Matriz de Riscos

ID	Risco	Probabilidade	Impacto	Mitigação
R-01	Backend sem deploy em nuvem: usuários não conseguem usar o app em produção	Alta	Crítico	Prioridade #1 do roadmap: deploy no Railway/Render na Fase 1
R-02	Ausência de RLS: vazamento de dados entre usuários via SQL injection ou bug de lógica	Média	Crítico	Ativar RLS no Supabase imediatamente após deploy; auditoria de queries
R-03	Sem recuperação de senha: usuário perde acesso permanentemente	Alta	Alto	Implementar fluxo de reset via Supabase Auth (Fase 1, Sprint 1)
R-04	GitHub Pages tem limitações de SPA routing (404 em refresh)	Média	Médio	Configurar 404.html redirect hack ou migrar para Netlify/Vercel
R-05	Supabase free tier: limite de 500 MB de storage e 50.000 auth users	Baixa	Médio	Monitorar uso; planejar upgrade para Pro tier se atingir 80% do limite
R-06	Sem testes automatizados: regressões passam despercebidas	Alta	Alto	Adicionar Jest + Supertest para rotas críticas na Fase 1
R-07	Dependência de terceiro (Supabase): se sair do ar, app inteiro para	Baixa	Crítico	Arquitetura stateless facilita migração; manter plano de contingência documentado
R-08	Leak de credenciais no repositório GitHub	Baixa	Crítico	.env no .gitignore; usar GitHub Secrets; ativar secret scanning no repo
R-09	Performance do banco sem índices adequados à medida que dados crescem	Média	Médio	Adicionar índices em user_id, study_area_id e created_at nas tabelas principais
R-10	UX mobile insatisfatória: Tailwind responsivo mas não testado em dispositivos reais	Média	Médio	Testes manuais em Chrome DevTools e dispositivos físicos antes do lançamento

9.2 Plano de Contingência — Supabase Outage

- Configurar health check endpoint no backend que valida conexão com Supabase
- Exibir página de manutenção amigável se Supabase retornar erro 5xx
- Documentar processo de migração para Auth0 + Neon/PlanetScale como alternativas
- Manter contato com status.supabase.com em canais de monitoramento

9.3 Plano de Contingência — Deploy de Backend

- Railway e Render como opções primária e secundária (configuração espelhada)
- Variáveis de ambiente documentadas no README (sem valores reais)
- Script de seed do banco para recriar ambiente rapidamente

Apêndice — Referência de API

Endpoint	Método	Auth	Descrição
GET /api/auth/me	GET	JWT	Retorna perfil do usuário autenticado
GET /api/dashboard	GET	JWT	Métricas agregadas: streak, horas, heatmap, resumo por área
GET /api/areas	GET	JWT	Lista todas as áreas de estudo do usuário
POST /api/areas	POST	JWT	Cria nova área de estudo
PUT /api/areas/:id	PUT	JWT	Atualiza área de estudo pelo ID
DELETE /api/areas/:id	DELETE	JWT	Remove área e todos os registros filhos (cascade)
GET /api/topics?area_id=x	GET	JWT	Lista tópicos de uma área
POST /api/topics	POST	JWT	Cria novo tópico
PUT /api/topics/:id	PUT	JWT	Atualiza tópico
DELETE /api/topics/:id	DELETE	JWT	Remove tópico
GET /api/tasks?area_id=x	GET	JWT	Lista tarefas de uma área
POST /api/tasks	POST	JWT	Cria nova tarefa
PUT /api/tasks/:id	PUT	JWT	Atualiza tarefa (inclui mudança de status)
DELETE /api/tasks/:id	DELETE	JWT	Remove tarefa
GET /api/materials?area_id=x	GET	JWT	Lista materiais de uma área
POST /api/materials	POST	JWT	Adiciona novo material
PUT /api/materials/:id	PUT	JWT	Atualiza material
DELETE /api/materials/:id	DELETE	JWT	Remove material
GET /api/sessions?area_id=x	GET	JWT	Lista sessões de estudo de uma área
POST /api/sessions	POST	JWT	Registra nova sessão de estudo
DELETE /api/sessions/:id	DELETE	JWT	Remove sessão de estudo